

VedaAI – Product Design & Requirements Document (PDR)

Role: Full Stack Engineer

Stack: Next.js + TypeScript | Node.js + Express | MongoDB | Redis | BullMQ | Gemini 2.5 Flash | WebSocket

Version: 1.0 | Complete Build Reference

TABLE OF CONTENTS

1. [Project Overview](#)
 2. [Complete User Flow — Step by Step](#)
 3. [Frontend Architecture](#)
 4. [State Management — Zustand](#)
 5. [Backend Architecture](#)
 6. [Database Design — MongoDB](#)
 7. [Queue System — BullMQ + Redis](#)
 8. [WebSocket Implementation](#)
 9. [AI Implementation — Gemini 2.5 Flash](#)
 10. [PDF Parsing](#)
 11. [Diagram & Visual Feature](#)
 12. [Output Page — Exam Paper Template](#)
 13. [PDF Export](#)
 14. [Caching Strategy](#)
 15. [LocalStorage Strategy](#)
 16. [Worker Definitions](#)
 17. [API Endpoints Reference](#)
 18. [Tech Stack Summary](#)
 19. [Folder Structure](#)
 20. [Environment Variables](#)
-

1. PROJECT OVERVIEW

VedaAI is an AI-powered assessment creator for teachers. A teacher provides metadata about the assessment (school, class, subject, chapter), configures the question structure (types, counts, marks), optionally uploads material (PDF or image), and the system generates a fully structured, exam-formatted question paper using Gemini 2.5 Flash. The output renders as a clean, printable exam paper and can be downloaded as PDF.

Core Capabilities

- Multi-step assignment creation form
 - AI-driven structured question paper generation via background worker
 - Real-time generation status via WebSocket
 - Structured JSON output rendered into exam paper template
 - Per-section regeneration (token-efficient)
 - Diagram support: SVG for simple shapes, Dagre.js for graphs/cycles, teacher-uploaded images
 - PDF export via Puppeteer
-

2. COMPLETE USER FLOW

2.1 Landing Page

- Hero section explaining the product
 - Two CTAs: **Login** and **Register**
 - No authenticated content visible
-

2.2 Registration

Fields collected:

- Full Name
- Email
- Password (min 8 chars, one number)
- School Name
- City / Location

Avatar Selection:

- After form submit, teacher is shown a grid of 8–10 predefined SVG avatars
- Teacher selects one
- Avatar ID (e.g., `avatar_03`) stored in DB, not the image itself

On success: JWT token issued, stored in `httpOnly` cookie, redirect to Homepage.

2.3 Login

- Email + Password
 - On success: JWT issued, redirect to Homepage
-

2.4 Homepage (Figma Screen 1 — matches sidebar layout)

- Sidebar: VedaAI logo, Create Assignment CTA, nav items (Home, My Groups, Assignments, AI Teacher's Toolkit, My Library), Settings, Teacher profile card at bottom
 - Main area: Welcome message, quick stats (total assignments, recent activity)
 - **No assignments yet state:** Shows the empty illustration with "No assignments yet" (matches Figma 0-state screen)
-

2.5 Assignments Page (Figma Screen 2)-- this is for reponsiveness

- Grid of assignment cards (2-column desktop, 1-column mobile)
 - Each card shows: Assignment title, Assigned on date, Due date, 3-dot menu (View Assignment / Delete)
 - Search bar + Filter by at top
 - Floating "+ Create Assignment" button at bottom center
 - **0-state:** Empty illustration (matches Figma exactly)
-

2.6 Create Assignment — Multi-Step Form

The form is a stepper with 2 steps. All data persisted to Zustand + LocalStorage at every change.

STEP 1 — Assessment Context

Fields:

- Assignment Title
- School Name (pre-filled from profile, editable)
- Class / Grade (dropdown: 1–12)
- Subject (text input)

- Chapter(s) / Topics (text input, comma-separated)
- Board (CBSE / ICSE / State Board / Other)
- Time Allowed (in minutes)

Validation: All fields required. No empty strings.

On "Next" → save to Zustand, navigate to Step 2.

STEP 2 — Question Configuration (matches Figma Screen 3)

Top section:

- File Upload zone (drag & drop or browse)
 - Accepts: PDF, JPEG, PNG, up to 10MB
 - On upload: file stored temporarily, upload to backend `/api/upload`, receive back a `fileRef` (S3 URL or local path)
- Due Date (date picker, DD-MM-YYYY format)

Question Type Builder:

- Dynamic list of question type rows (boxes type input form structure)
- Each row has:
 - Question Type dropdown: Multiple Choice Questions / Short Answer / Long Answer / Diagram/Graph-Based / Numerical Problems / True-False
 - Number of Questions (stepper, min 1)
 - Marks per question (stepper, min 1)
 - Delete row (×)
- "+ Add Question Type" button adds a new row
- Running totals shown at bottom: **Total Questions: N | Total Marks: N**

Additional Instructions:

- Textarea: e.g., "Generate for 3-hour exam, include HOTS questions"

Validation:

- At least one question type row
- No zero or negative values
- Due date must be in future

On "Generate Assessment" click:

1. Validate all fields
2. Save complete form state to Zustand

3. POST to `/api/assignments` → receive `assignmentId` + `jobId`
 4. Open WebSocket connection, subscribe to `jobId`
 5. Navigate to `/assignments/[assignmentId]/generating` — loading screen
-

2.7 Generation Loading Screen

- Shows assignment title
 - Animated status messages driven by WebSocket events:
 - "Initializing generation..."
 - "Analyzing your content..."
 - "Generating Section A..."
 - "Generating Section B..."
 - "Finalizing structure..."
 - "Done!"
 - Progress bar updates per section received
 - If error: show retry button
-

2.8 Output Page — Exam Paper (Figma Screen 4)

Top banner (dark):

- AI confirmation message: "Here are your customized questions for [Subject], [Class]"
- Download as PDF button
- Regenerate button (triggers full regeneration)

Exam Paper (white card, A4-like):

- School Name (bold, centered, large)
- Subject | Class (centered)
- Time Allowed | Maximum Marks (two columns)
- Horizontal rule
- Instructions line
- Student info lines: Name ____ Roll Number ____ Section ____
- Horizontal rule
- Sections (A, B, C...) rendered in order
 - Section title (bold, centered)
 - Section instruction (italic)
 - Numbered question list
 - Question text
 - Difficulty badge (Easy = green, Moderate = orange, Hard/Challenging = red)

- Marks tag [X Marks]
- MCQ options if type = MCQ
- Image if question has `imageUrl`
- Diagram if question has `diagramData`
- End of Question Paper (centered, bold)
- Answer Key section (collapsible / below a divider)

Per-section regenerate:

- Each section has a small "Regenerate" icon button
- Clicking it triggers a targeted regeneration job for only that section

2.9 My Library Page

- Organized by Subject
- Each subject is a collapsible group
- Inside each group: list of assessment cards for that subject
- Each card: title, class, date created, view button

3. FRONTEND ARCHITECTURE

Framework: Next.js 14 App Router + TypeScript

Pages / Routes:

/	→ Landing page
/auth/login	→ Login
/auth/register	→ Register
/auth/register/avatar	→ Avatar selection
/dashboard	→ Homepage
/assignments	→ Assignments list
/assignments/create	→ Multi-step form
/assignments/[id]/generating	→ Loading screen
/assignments/[id]/output	→ Exam paper output
/library	→ My Library

Component Structure:

```
components/
  layout/
```

- Sidebar.tsx
- TopBar.tsx
- MobileNav.tsx
- form/
 - StepOne.tsx
 - StepTwo.tsx
 - QuestionTypeRow.tsx
 - FileUploadZone.tsx
- output/
 - ExamPaper.tsx
 - SectionBlock.tsx
 - QuestionItem.tsx
 - DifficultyBadge.tsx
 - DiagramRenderer.tsx
 - MCQOptions.tsx
- common/
 - Avatar.tsx
 - AssignmentCard.tsx
 - StatusBar.tsx

4. STATE MANAGEMENT — ZUSTAND

Two stores:

Store 1: **useAssignmentFormStore**

```
interface AssignmentFormStore {  
  // Step 1  
  title: string  
  schoolName: string  
  className: string  
  subject: string  
  chapters: string  
  board: string  
  timeAllowed: number  
  
  // Step 2  
  dueDate: string  
  questionTypes: QuestionTypeRow[]  
  additionalInstructions: string  
  fileRef: string | null    // URL returned after upload  
  fileType: 'pdf' | 'image' | null
```

```

// Meta
currentStep: number
isDirty: boolean

// Actions
setField: (field, value) => void
addQuestionTypeRow: () => void
removeQuestionTypeRow: (id) => void
updateQuestionTypeRow: (id, data) => void
resetForm: () => void
}

```

Persisted to LocalStorage via Zustand persist middleware. Key: `veda_form_draft`

Cleared: Only when teacher explicitly starts a new assignment after successful generation.

Store 2: `useGenerationStore`

```

interface GenerationStore {
  jobId: string | null
  assignmentId: string | null
  status: 'idle' | 'queued' | 'processing' | 'completed' | 'failed'
  progress: number // 0–100
  currentSectionLabel: string // "Generating Section B..."
  completedSections: AssessmentSection[] // sections as they arrive
  error: string | null

  // Actions
  setJobId: (id) => void
  addSection: (section) => void
  setStatus: (status) => void
  setError: (msg) => void
  reset: () => void
}

```

Not persisted. Lives only in memory for the current generation session.

5. BACKEND ARCHITECTURE

Runtime: Node.js + Express + TypeScript

Entry: `src/index.ts` → Express app + WebSocket server on same port using `ws` library

Middleware Stack

- `cors` with frontend origin whitelist
- `helmet` for security headers
- `express.json()` with 10mb limit
- JWT auth middleware on all `/api/*` routes except `/api/auth/*`
- Rate limiter on `/api/assignments/generate`

Service Layers

`src/`

`routes/` → Express route handlers (thin, just validates and delegates)

`services/` → Business logic

`assignmentService.ts`

`generationService.ts`

`pdfService.ts`

`uploadService.ts`

`workers/` → BullMQ worker definitions

`generationWorker.ts`

`pdfWorker.ts`

`queues/` → BullMQ queue definitions

`generationQueue.ts`

`pdfQueue.ts`

`websocket/` → WebSocket server and event emitter

`wsServer.ts`

`wsEvents.ts`

`ai/` → Gemini integration

`geminiClient.ts`

`promptBuilder.ts`

`responseParser.ts`

`zodSchemas.ts`

`db/` → MongoDB models

`cache/` → Redis helpers

6. DATABASE DESIGN — MONGODB

Collection: `users`

{

```

_id: ObjectId,
name: string,
email: string,      // unique
passwordHash: string,
schoolName: string,
city: string,
avatarId: string,   // e.g. "avatar_03"
createdAt: Date
}

```

Collection: **assignments**

```

{
  _id: ObjectId,
  userId: ObjectId,  // ref: users
  title: string,
  schoolName: string,
  className: string,
  subject: string,
  chapters: string,
  board: string,
  timeAllowed: number, // minutes
  dueDate: Date,
  questionTypes: [
    {
      type: string,    // "MCQ" | "Short Answer" etc.
      count: number,
      marksEach: number
    }
  ],
  additionalInstructions: string,
  fileRef: string | null, // S3/local URL of uploaded material
  fileType: string | null,
  status: 'pending' | 'processing' | 'completed' | 'failed',
  createdAt: Date,
  updatedAt: Date
}

```

Collection: **generatedPapers**

```

{
  _id: ObjectId,
  assignmentId: ObjectId, // ref: assignments

```

```

    userId: ObjectId,
    paper: AssessmentJSON, // full structured JSON (schema defined below)
    generatedAt: Date,
    version: number        // increments on full regeneration
}

```

AssessmentJSON Schema (the core data structure)

```

interface AssessmentJSON {
  title: string
  schoolName: string
  subject: string
  className: string
  board: string
  timeAllowed: number
  totalMarks: number
  totalQuestions: number
  instructions: string
  sections: AssessmentSection[]
}

```

```

interface AssessmentSection {
  id: string          // "A", "B", "C"
  title: string        // "Section A"
  questionTypeName: string // "Short Answer Questions"
  instruction: string   // "Attempt all questions. Each carries 2 marks."
  questions: Question[]
}

```

```

interface Question {
  id: number          // global sequential number
  text: string
  type: 'MCQ' | 'short' | 'long' | 'diagram' | 'numerical' | 'truefalse'
  difficulty: 'easy' | 'moderate' | 'hard'
  marks: number
  options?: string[]  // MCQ only, array of 4 strings
  answer?: string      // Answer key
  imageUrl?: string    // Teacher-uploaded image URL
  diagramData?: DiagramData | null
}

```

```

interface DiagramData {
  renderType: 'svg' | 'dagre'
  // If renderType === 'svg':

```

```
svgContent?: string    // Raw SVG string (sanitized)
// If renderType === 'dagre':
nodes?: { id: string; label: string }[]
edges?: { from: string; to: string; label?: string }[]
}
```

7. QUEUE SYSTEM — BULLMQ + REDIS

Why BullMQ Here

Gemini API calls take 10–40 seconds. PDF generation with Puppeteer takes 5–15 seconds. Both would cause HTTP timeouts if run synchronously in Express routes. BullMQ decouples the request from the processing.

Redis acts as BullMQ's backing store — it holds the job payloads, tracks job states (waiting → active → completed/failed), and enables retry logic.

Queue 1: **assessment-generation**

Added when: Teacher clicks "Generate Assessment", POST `/api/assignments` is called.

Job payload:

```
{
  assignmentId: string,
  userId: string,
  title: string,
  schoolName: string,
  className: string,
  subject: string,
  chapters: string,
  board: string,
  timeAllowed: number,
  questionTypes: QuestionTypeRow[],
  additionalInstructions: string,
  fileRef: string | null,
  fileType: 'pdf' | 'image' | null
}
```

Worker behavior (step by step):

1. Worker picks up job
2. Emit WebSocket event: `{ status: 'processing', message: 'Analyzing your content...' }`
3. If `fileRef` exists and `fileType === 'pdf'`: parse PDF with `pdf-parse`, extract text
4. If `fileRef` exists and `fileType === 'image'`: read image as base64
5. Build system prompt (see Section 9)
6. Call Gemini 2.5 Flash with structured output prompt
7. Stream response section by section
8. For each completed section: validate with Zod, emit WebSocket `section_ready` event
9. After all sections: validate full JSON with Zod
10. Save to `generatedPapers` collection in MongoDB
11. Update `assignments` status to `completed`
12. Cache full JSON in Redis: key `paper:{assignmentId}`, TTL 2 hours
13. Emit WebSocket: `{ status: 'completed', assignmentId }`

On failure:

- BullMQ retries up to 2 times with exponential backoff
- After all retries exhausted: update assignment status to `failed`, emit WebSocket `{ status: 'failed', error: '...' }`

Concurrency: 3 concurrent workers (Gemini rate limit safe)

Queue 2: `section-regeneration`

Added when: Teacher clicks the regenerate icon on a specific section.

Job payload:

```
{
  assignmentId: string,
  userId: string,
  sectionId: string,    // "A", "B", etc.
  existingPaper: AssessmentJSON, // full current paper (other sections preserved)
  // same context fields as above
}
```

Worker behavior:

1. Fetch existing paper from Redis cache (or MongoDB if cache miss)
2. Build targeted prompt: only regenerate the specified section, same context
3. Call Gemini, get only that section's questions
4. Validate section with Zod
5. Merge new section into existing paper JSON (replace only that section)
6. Update MongoDB `generatedPapers` document
7. Update Redis cache
8. Emit WebSocket: `{ status: 'section_updated', sectionId, section: newSection }`

Why this is token-efficient: You're only asking Gemini to produce ~5–10 questions instead of the full paper. Saves roughly 60–80% of tokens on a regeneration call.

Queue 3: `pdf-export`

Added when: Teacher clicks "Download as PDF".

Job payload:

```
{
  assignmentId: string,
  userId: string,
  paperId: string
}
```

Worker behavior:

1. Fetch paper JSON from Redis (or MongoDB)
 2. Render the exam paper HTML template server-side (same template as frontend output page, but a server-rendered HTML string)
 3. Launch Puppeteer, load HTML string
 4. Set page format to A4, print to PDF buffer
 5. Save PDF temporarily to `/tmp/[assignmentId].pdf`
 6. Respond to frontend with download stream or signed URL
 7. Delete temp file after delivery
-

8. WEBSOCKET IMPLEMENTATION

Library: `ws` (Node.js WebSocket library)

Setup: WebSocket server attached to the same Express HTTP server instance

Server Setup

Express HTTP Server

└─ WebSocket Server (`ws`)

└─ Connection map: `Map<jobId, WebSocket>`

When client connects, it sends an init message: `{ type: 'subscribe', jobId: 'xyz' }`. Server stores `connectionMap.set(jobId, ws)`.

Event Types Emitted by Server → Client

Event	Payload	Triggered by
<code>status_update</code>	<code>{ status, message }</code>	Worker progress
<code>section_ready</code>	<code>{ sectionId, section: AssessmentSection }</code>	Each section completed
<code>generation_complete</code>	<code>{ assignmentId }</code>	All sections done
<code>section_updated</code>	<code>{ sectionId, section }</code>	Section regeneration done
<code>generation_failed</code>	<code>{ error: string }</code>	Job failed after retries
<code>pdf_ready</code>	<code>{ downloadUrl }</code>	PDF job complete

Client WebSocket Handler (frontend)

// In generating page component

```
useEffect(() => {  
  const ws = new WebSocket(process.env.NEXT_PUBLIC_WS_URL)  
  
  ws.onopen = () => {  
    ws.send(JSON.stringify({ type: 'subscribe', jobId }))  
  }  
  
  ws.onmessage = (event) => {  
    const msg = JSON.parse(event.data)
```

```
switch(msg.type) {
  case 'section_ready':
    generationStore.addSection(msg.section)
    break
  case 'generation_complete':
    router.push(`/assignments/${assignmentId}/output`)
    break
  case 'generation_failed':
    generationStore.setError(msg.error)
    break
}
}

return () => ws.close()
}, [jobId])
```

Why WebSocket Over Polling

A single persistent connection. The server pushes sections the moment they're ready. With polling (GET every 2s), you'd make 10–20 requests during a 20-second generation. WebSocket delivers each section the instant it completes — enabling the streaming section-by-section reveal UX on the output page.

9. AI IMPLEMENTATION — GEMINI 2.5 FLASH

Model: `gemini-2.5-flash`

Library: `@google/generative-ai`

Why Gemini 2.5 Flash

- Free tier available
 - Strong structured JSON output
 - Long context window (handles large PDFs)
 - Fast enough for streaming section delivery
-

Prompt Architecture

Every generation call has two parts: **System Context** and **User Instruction**.

System Context (built by **promptBuilder.ts**)

You are an expert educational assessment creator for Indian schools.
You create structured, curriculum-aligned question papers.

SCHOOL CONTEXT:

- School: {schoolName}
- Class: {className}
- Subject: {subject}
- Chapters/Topics: {chapters}
- Board: {board}
- Time Allowed: {timeAllowed} minutes

STRICT RULES:

1. Questions must be appropriate for Class {className} students
2. Vocabulary, complexity, and depth must match {board} curriculum standards
3. You must return ONLY valid JSON. No markdown, no preamble, no explanation.
4. Follow the exact JSON schema provided.
5. Difficulty distribution per section: 40% easy, 40% moderate, 20% hard unless instructed otherwise.
6. MCQ options must be plausible — wrong options should not be obviously wrong.

User Instruction

Generate a complete question paper with the following structure:

SECTIONS REQUIRED:

{foreach questionType row}

- Section {letter}: {count} {type} questions, {marksEach} marks each
- {/foreach}

ADDITIONAL INSTRUCTIONS: {additionalInstructions}

{if pdfText}

BASE YOUR QUESTIONS ON THIS CONTENT:

{extractedPDFText}

{/if}

RESPOND ONLY WITH THIS JSON STRUCTURE:

{paste full AssessmentJSON schema with field descriptions}

Response Parsing (**responseParser.ts**)

1. Receive raw string from Gemini
 2. Strip any accidental markdown fences (**json ...**)
 3. **JSON.parse()**
 4. Validate against Zod schema
 5. If Zod fails: log the error, retry once with a corrective prompt
 6. If second attempt fails: throw error, let BullMQ handle retry
-

AI Scenario: Section Regeneration

System context is the same. User instruction changes:

I have an existing question paper. Regenerate ONLY Section {sectionId} ({questionTypeName}).
Keep the same difficulty distribution and marks structure.
The rest of the paper is unchanged.

SECTION TO REGENERATE:

- Type: {questionTypeName}
- Count: {count} questions
- Marks each: {marksEach}

Return ONLY the section JSON in this format:

```
{ "section": AssessmentSection }
```

AI Scenario: Diagram Generation (Dagre)

Called separately when a question's type is **diagram** or **graph**:

Generate a simple directed graph/cycle diagram for the following concept:
"{questionText}"

Return ONLY this JSON:

```
{  
  "renderType": "dagre",  
  "nodes": [{ "id": "1", "label": "Node Name" }],  
  "edges": [{ "from": "1", "to": "2", "label": "optional edge label" }]  
}
```

Keep it simple: maximum 8 nodes, clear labels, school-level accuracy.

AI Scenario: Simple Shape Diagram (SVG)

For geometric or simple labeled diagrams where Dagre doesn't fit:

Generate a simple SVG diagram for: "{diagramDescription}"

Rules:

- Use only basic SVG elements: circle, rect, line, text, path
- Black and white only (stroke: black, fill: white or none)
- ViewBox: 0 0 300 300
- All text must be inside <text> elements with appropriate font-size
- No external resources, no JavaScript, no CSS classes
- Keep it simple and school-appropriate

Return ONLY the raw SVG string starting with <svg>

After receiving: sanitize with DOMPurify before storing in DB or rendering.

10. PDF PARSING

Library: `pdf-parse`

When used: When teacher uploads a PDF in Step 2 of the form.

Flow:

1. Teacher uploads PDF via file upload zone
2. Frontend sends multipart form POST to `/api/upload`
3. Backend receives file buffer (Multer middleware)
4. Save file to local `/uploads/` directory or S3
5. Return `fileRef` URL to frontend
6. Zustand stores `fileRef + fileType: 'pdf'`
7. When generation job runs: worker fetches the file, runs `pdf-parse`
8. Extracted text (plain string) is trimmed and truncated to 15,000 characters (Gemini context limit safety)
9. Injected into the generation prompt under "BASE YOUR QUESTIONS ON THIS CONTENT"

Important: Do not pass the raw file to Gemini. Extract text first. Cleaner, cheaper, more reliable.

If image uploaded (JPEG/PNG):

- Worker reads the file as base64
 - Passes it to Gemini as an inline image part in the multimodal message
 - Gemini can read text and diagrams from the image
-

11. DIAGRAM AND VISUAL FEATURE

Three rendering paths, chosen by the AI response field `diagramData.renderType`.

Path 1: Teacher-Uploaded Image

- Teacher uploads image via question form
- Image stored on backend, `imageUrl` stored in question JSON
- Frontend `QuestionItem` renders `<img src={imageUrl} />` with max-width constraint
- In PDF: Puppeteer captures it naturally as part of the page

Path 2: SVG (Simple Shapes)

- Used for: geometric figures, labeled diagrams (e.g., parts of a plant), simple science diagrams
- Gemini generates raw SVG string
- Backend sanitizes with `isomorphic-dompurify` before storing
- Frontend `DiagramRenderer` renders `<div dangerouslySetInnerHTML={{ __html: svgContent }} />`
- Constrained inside a fixed 300×300 container

Path 3: Dagre.js (Graphs, Cycles, Flowcharts)

- Used for: food chains, water cycle, flowcharts, process diagrams, organizational structures
- Gemini returns nodes + edges JSON (NOT coordinates)
- Frontend `DiagramRenderer` uses `dagre` library to compute layout
- Renders nodes as `<rect>` + `<text>`, edges as `<line>` or `<path>` with arrowheads
- All inside a dynamically sized SVG element

DiagramRenderer Decision Logic:

```
if (!question.diagramData && !question.imageUrl) return null
if (question.imageUrl) return <img src={question.imageUrl} />
if (question.diagramData.renderType === 'svg') return <SvgDiagram />
if (question.diagramData.renderType === 'dagre') return <DagreDiagram />
```

12. OUTPUT PAGE — EXAM PAPER TEMPLATE

The output is a React component that renders in the style of a real Indian school exam paper. It is a fixed template — the data fills it, the structure is predetermined.

Visual Structure (matches Figma Screen 4)

[Dark Banner]

"Here are your customized questions for..."

[Download PDF] [Regenerate All]

DELHI PUBLIC SCHOOL, BOKARO ← bold, centered, large

Subject: Science | Class: 8th ← centered

Time Allowed: 45 mins Maximum Marks: 50

All questions are compulsory unless stated otherwise.

Name: _____ Roll No: _____

Class/Section: _____

SECTION A ← bold, centered

Short Answer Questions ← centered

Attempt all questions. Each carries 2 marks. ← , smaller

1. [Easy] Question text here... [2 Marks]

2. [Moderate] Question text here... [2 Marks]

[Diagram here if applicable]

SECTION B

Multiple Choice Questions

Choose the correct answer.

11. [Easy] Question text... [1 Mark]

(a) Option A	(b) Option B	
(c) Option C	(d) Option D	
END OF QUESTION PAPER		
ANSWER KEY		
1. Answer text...		
2. Answer text...		

Difficulty Badge Colors

- Easy → green background (**#dcfce7**), green text
- Moderate → orange background (**#fff7ed**), orange text
- Hard/Challenging → red background (**#fef2f2**), red text

Question Numbering

Questions numbered globally across all sections (1, 2, 3... continues across sections). Section breaks are visual separators, not number resets.

Mobile Responsive

- On mobile: paper fills full width, font scales down slightly
- Difficulty badges remain inline with question
- MCQ options stack to 2-per-row instead of 2-per-line

13. PDF EXPORT

Library: Puppeteer (backend, in PDF worker)

Flow:

1. Teacher clicks "Download as PDF"
2. POST **/api/assignments/:id/export-pdf**
3. Job added to **pdf-export** queue
4. Worker launches Puppeteer headless browser
5. Loads a server-rendered HTML version of the exam paper (same CSS, same structure)
6. Sets Puppeteer print options: A4, margins 15mm all sides, background graphics enabled

7. Generates PDF buffer
8. Streams back to client as `application/pdf` download
9. Filename: `{SchoolName}_{Subject}_{Class}_{Date}.pdf`

Why Puppeteer over LLM-generated PDF: Puppeteer renders exactly what you see on screen. Pixel-perfect. No formatting discrepancies.

Key Puppeteer config:

```
await page.emulateMediaType('print')
const pdf = await page.pdf({
  format: 'A4',
  printBackground: true,
  margin: { top: '15mm', right: '15mm', bottom: '15mm', left: '15mm' }
})
```

14. CACHING STRATEGY

Redis Cache Keys and TTLs

Key Pattern	Value	TTL	Purpose
<code>paper:{assignmentId}</code>	Full AssessmentJSON (JSON string)	2 hours	Avoid DB hit on output page load
<code>job:{jobId}:status</code>	<code>{ status, progress, message }</code>	1 hour	WebSocket reconnect recovery
<code>user:{userId}:assignments</code>	Array of assignment summaries	10 minutes	Assignments list page

Cache Invalidation Rules

- `paper:{assignmentId}` is invalidated and rewritten whenever a section is regenerated or full regeneration runs
- `user:{userId}:assignments` is invalidated whenever an assignment is created or deleted
- Never cache PDF binary in Redis — too large, unnecessary

Cache Miss Handling

Every cache read has a fallback:

Redis GET → if null → MongoDB query → write result back to Redis → return

15. LOCALSTORAGE STRATEGY

Managed by: Zustand `persist` middleware

Key: `veda_form_draft`

What is Stored

- All Step 1 form fields (title, class, subject, chapters, board, time)
- All Step 2 form fields (due date, question type rows, additional instructions)
- Current step number (1 or 2)
- `fileRef` of uploaded file (so teacher doesn't need to re-upload on refresh)

What is NOT Stored in LocalStorage

- JWT token (stored in httpOnly cookie only)
- Generated paper JSON (stored in MongoDB + Redis, fetched on output page load)
- Zustand `generationStore` (ephemeral, session-only)

Clearing LocalStorage

- On successful generation completion (after teacher views output): `resetForm()` called in store
 - On explicit "Start New Assignment": `resetForm()` called
 - Never cleared on error — teacher should be able to resume
-

16. WORKER DEFINITIONS

Worker 1: `generationWorker.ts`

Queue: `assessment-generation`

Concurrency: 3

Steps:

1. Extract job data

2. Parse uploaded file if present (pdf-parse or base64 image)
3. Build system prompt using `promptBuilder.ts`
4. Call Gemini, stream response
5. Parse + validate each section with Zod
6. Emit `section_ready` WebSocket event per section
7. Save complete paper to MongoDB
8. Cache in Redis
9. Update assignment status
10. Emit `generation_complete` WebSocket event

Retry config: `{ attempts: 2, backoff: { type: 'exponential', delay: 5000 } }`

Worker 2: `sectionRegenerationWorker.ts`

Queue: `section-regeneration`

Concurrency: 5

Steps:

1. Fetch existing paper from Redis (fallback MongoDB)
2. Build targeted section-only prompt
3. Call Gemini
4. Parse + validate section with Zod
5. Merge new section into existing paper
6. Update MongoDB + Redis cache
7. Emit `section_updated` WebSocket event

Retry config: `{ attempts: 2, backoff: { type: 'fixed', delay: 3000 } }`

Worker 3: `pdfWorker.ts`

Queue: `pdf-export`

Concurrency: 2

Steps:

1. Fetch paper JSON from Redis/MongoDB
2. Render HTML string from exam paper template
3. Launch Puppeteer, load HTML
4. Generate PDF buffer

5. Stream to client or save to `/tmp/` and send signed URL
6. Cleanup temp file

Retry config: `{ attempts: 1 }` (PDF failures should surface immediately)

17. API ENDPOINTS

Auth

Method	Path	Description
POST	<code>/api/auth/register</code>	Create teacher account
POST	<code>/api/auth/login</code>	Login, returns JWT in cookie
POST	<code>/api/auth/logout</code>	Clear cookie

Assignments

Method	Path	Description
GET	<code>/api/assignments</code>	List all assignments for teacher
POST	<code>/api/assignments</code>	Create assignment + add to generation queue
GET	<code>/api/assignments/:id</code>	Get single assignment metadata
DELETE	<code>/api/assignments/:id</code>	Delete assignment + paper
GET	<code>/api/assignments/:id/paper</code>	Get generated paper JSON (Redis → MongoDB)

Generation

Method	Path	Description
GET	<code>/api/assignments/:id/status</code>	Get job status (fallback for WebSocket reconnect)

POST	<code>/api/assignments/:id/regenerate</code>	Full regeneration
POST	<code>/api/assignments/:id/regenerate-section</code>	Section regeneration

Export

Method	Path	Description
POST	<code>/api/assignments/:id/export-pdf</code>	Queue PDF generation
GET	<code>/api/assignments/:id/download</code>	Stream PDF

Upload

Method	Path	Description
POST	<code>/api/upload</code>	Upload PDF/image, return fileRef URL

18. TECH STACK

Frontend

Tool	Purpose
Next.js 14 (App Router)	Framework, routing, SSR
TypeScript	Type safety
Zustand + persist middleware	State management + LocalStorage
TailwindCSS	Styling
React Hook Form + Zod	Form validation
dagre	Graph layout computation

ws (browser WebSocket)	Real-time updates
axios	HTTP client

Backend

Tool	Purpose
Node.js + Express + TypeScript	Server
ws	WebSocket server
BullMQ	Job queue
iredis	Redis client
Mongoose	MongoDB ODM
Multer	File upload handling
pdf-parse	PDF text extraction
isomorphic-dompurify	SVG sanitization
Puppeteer	PDF export
jsonwebtoken	Auth tokens
bcryptjs	Password hashing
zod	Runtime JSON validation

AI

Tool	Purpose
@google/generative-ai	Gemini 2.5 Flash SDK
Gemini 2.5 Flash	Question generation, diagram data

Infrastructure

Tool	Purpose
------	---------

MongoDB (Atlas free tier)	Persistent storage
Redis (Upstash free tier)	Cache + BullMQ backing
Vercel	Frontend deployment
Railway / Render	Backend deployment

19. FOLDER STRUCTURE

```

vedaai/
├── frontend/                                # Next.js app
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   ├── register/page.tsx
│   │   │   └── register/avatar/page.tsx
│   │   ├── dashboard/page.tsx
│   │   ├── assignments/
│   │   │   ├── page.tsx
│   │   │   ├── create/page.tsx
│   │   │   └── [id]/
│   │   │       ├── generating/page.tsx
│   │   │       └── output/page.tsx
│   │   └── library/page.tsx
│   ├── components/
│   │   ├── layout/
│   │   ├── form/
│   │   ├── output/
│   │   └── common/
│   ├── stores/
│   │   ├── assignmentFormStore.ts
│   │   └── generationStore.ts
│   └── lib/
│       ├── api.ts
│       └── websocket.ts
└── backend/                                # Express app
    ├── src/
    │   ├── index.ts
    │   ├── routes/
    │   ├── services/
    │   └── workers/

```

```

├── generationWorker.ts
├── sectionRegenerationWorker.ts
├── pdfWorker.ts
├── queues/
│   ├── generationQueue.ts
│   └── pdfQueue.ts
├── websocket/
│   └── wsServer.ts
├── ai/
│   ├── geminiClient.ts
│   ├── promptBuilder.ts
│   ├── responseParser.ts
│   └── zodSchemas.ts
├── db/
│   ├── models/
│   │   ├── User.ts
│   │   ├── Assignment.ts
│   │   └── GeneratedPaper.ts
│   └── connect.ts
├── cache/
│   └── redis.ts
├── uploads/ # Temp upload directory

```

ADDENDUM — Implementation Details

1. Error Handling

Every layer has a defined error behavior. Nothing fails silently.

Frontend:

- Wrap the output page in a React Error Boundary. If any component throws, show a fallback UI with a "Reload" button — not a blank white screen.
- Every API call wrapped in try/catch. On network error: show a toast notification with the specific message, not a generic "Something went wrong."
- Form validation errors shown inline under the specific field, not as a page-level alert.
- If generation job fails (WebSocket delivers `generation_failed` event): show the error message with a "Try Again" button that re-queues the same job without making the teacher re-fill the form.

Backend:

- Global Express error handler as the last middleware. Catches anything that falls through and returns structured JSON: `{ success: false, error: { code, message } }`. Never leak stack traces to the client in production.
- All route handlers wrapped with an `asyncHandler` utility that catches async errors and passes them to the global handler — no unhandled promise rejections.
- Zod validation failure in worker: log the raw LLM response and the Zod error to console, retry once with a corrective prompt appended ("Your previous response was invalid JSON. Return only valid JSON matching this schema."). If second attempt also fails, mark job as failed and emit error via WebSocket.
- MongoDB connection failure on startup: exit process with code 1 and a clear log message. Do not start the server with no DB connection.
- Redis connection failure: log a warning but continue — degrade gracefully by bypassing cache (go directly to MongoDB). BullMQ will not start if Redis is unavailable, so log that workers are offline.

Worker-level:

- Each worker's `process` function wrapped in try/catch. On caught error: update assignment status to `failed` in MongoDB, emit `generation_failed` WebSocket event, let BullMQ handle the retry count.
 - After all retry attempts exhausted: emit final failure event with a human-readable message.
-

2. No Streaming — Full Response Generation

Generation is not streamed. The Gemini call runs to completion, the full JSON response is received, validated with Zod, saved to MongoDB, and only then the WebSocket emits a single `generation_complete` event with the full paper.

This simplifies the worker significantly — no partial JSON parsing, no chunk reassembly, no section-by-section emit logic.

Loading UX without streaming: Show a deterministic progress animation driven by time elapsed, not actual server events. A sequence of status messages cycles automatically on the frontend:

- 0s: "Submitting your assignment..."
- 2s: "Analyzing content and context..."
- 5s: "Building question structure..."
- 10s: "Generating questions..."
- 18s: "Reviewing and formatting..."
- On `generation_complete` event received: "Done! Loading your paper..."

These are frontend timers only — they do not reflect actual backend progress. But they keep the teacher engaged during the wait and are far simpler to implement than real streaming.

WebSocket in this model carries only three events:

- `generation_complete` — full paper is ready, `assignmentId` in payload
- `generation_failed` — job failed, error message in payload
- `section_updated` — a single section was regenerated (still used for section-level regeneration)

3. WebSocket — Volatile Delivery with Poll Fallback

WebSocket messages are fire-and-forget (volatile). If a packet is lost due to network congestion or a dropped connection, do not retry it. Move on.

Server behavior: Emit WebSocket events with no acknowledgement requirement. If the client is not connected when the event fires, the event is dropped. The source of truth is always MongoDB — WebSocket is just a notification channel, not a data delivery mechanism.

typescript

// In worker, after generation completes:

```
const ws = connectionMap.get(jobId)
if (ws && ws.readyState === WebSocket.OPEN) {
  ws.send(JSON.stringify({ type: 'generation_complete', assignmentId }))
}
```



```
}  
// If ws is null or closed — drop it. Client will poll.
```

Client reconnect + poll fallback: If the WebSocket connection drops during generation, the client falls back to polling automatically.

typescript

```
useEffect(() => {  
  let pollInterval: NodeJS.Timeout  
  
  const ws = new WebSocket(WS_URL)  
  
  ws.onopen = () => ws.send(JSON.stringify({ type: 'subscribe', jobId }))  
  
  ws.onmessage = (event) => {  
    const msg = JSON.parse(event.data)  
    if (msg.type === 'generation_complete') {  
      clearInterval(pollInterval)  
      router.push(`/assignments/${assignmentId}/output`)  
    }  
    if (msg.type === 'generation_failed') {  
      clearInterval(pollInterval)  
      setError(msg.error)  
    }  
  }  
  
  ws.onclose = () => {  
    // WebSocket dropped — start polling as fallback  
    pollInterval = setInterval(async () => {  
      const res = await fetch(`/api/assignments/${assignmentId}/status`)  
      const data = await res.json()  
      if (data.status === 'completed') {  
        clearInterval(pollInterval)  
        router.push(`/assignments/${assignmentId}/output`)  
      }  
      if (data.status === 'failed') {  
        clearInterval(pollInterval)  
        setError(data.error)  
      }  
    }, 3000)  
  }  
  
  return () => {  
    ws.close()  
    clearInterval(pollInterval)  
  }  
}
```

```
}  
}, [jobId, assignmentId])
```

Poll interval: every 3 seconds. Stop polling the moment either a status of **completed** or **failed** is returned. This covers every failure scenario — tab sleep, network blip, server restart — without any complex reconnection logic.

4. Optimistic UI on Section Regeneration

When the teacher clicks the regenerate icon on a section, do not wait for the WebSocket event before updating the UI. Immediately replace that section's content with a skeleton shimmer. This makes the interaction feel instant.

State change on click:

```
typescript  
// In generationStore, add:  
regeneratingSections: Set<string> // section IDs currently regenerating  
setRegenerating (sectionId: string, value: boolean) => void
```

On regenerate button click:

1. Immediately call `setRegenerating('B', true)` — UI swaps section content for skeleton
2. POST to `/api/assignments/:id/regenerate-section` with `{ sectionId: 'B' }`
3. Receive back a `jobId`
4. WebSocket is already connected — it will deliver `section_updated` when done

SectionBlock component render logic:

```
typescript  
const isRegenerating = regeneratingSections.has(section.id)  
  
if (isRegenerating) return <SectionSkeleton />  
return <SectionContent section={section} />
```

SectionSkeleton: Three to five grey shimmer bars of varying widths. Animating pulse (Tailwind `animate-pulse`). Looks like loading question text. Renders inside the same section container so the page layout does not shift.

On `section_updated` WebSocket event received:

1. Call `setRegenerating('B', false)`
2. Update the section data in the store
3. Section content re-renders with new questions

On failure:

1. Call `setRegenerating('B', false)`
2. Restore the previous section data (keep a `previousSections` snapshot in the store before triggering)
3. Show a toast: "Failed to regenerate Section B. Previous version restored."

APPENDIX: BUILD PRIORITY ORDER

Build in this exact order. Do not skip ahead.

1. **Auth** (register, login, JWT, avatar selection)
2. **Sidebar layout + Homepage** (match Figma exactly)
3. **Assignments list page** (0-state and filled state)
4. **Multi-step form** (Step 1 + Step 2, validation, Zustand + LocalStorage)
5. **File upload endpoint**
6. **MongoDB models + Redis setup**
7. **BullMQ queues + generation worker** (without AI first — mock the Gemini call)
8. **WebSocket server + client integration**
9. **Gemini integration** (replace mock with real call)
10. **Zod validation + response parsing**
11. **Output page** (exam paper template, all question types)
12. **Section regeneration** (worker + UI button)
13. **PDF export** (Puppeteer worker)
14. **Diagram rendering** (SVG + Dagre)
15. **My Library page**
16. **Polish, mobile responsive, README**

Color System

Primary brand color: #E8470A (the burnt orange/rust — used for the logo mark, CTA button background, active nav item background, badge on nav, notification dot, "Next" button)

Background colors:

- Page/sidebar background: #FFFFFF (pure white)
- Main content area: #F5F5F5 (very light grey, almost white)
- Assignment cards: #FFFFFF with a subtle border
- Top navbar: #FFFFFF with a bottom border #E5E5E5
- Dark banner (output page): #1A1A1A (near black)

Text colors:

- Primary headings: #111111 (near black)
- Secondary text: #6B6B6B (medium grey)
- Muted/label text: #9CA3AF (light grey)
- White text (on dark banner, on CTA button): #FFFFFF

Border colors:

- Card borders: #E5E7EB (1px solid)
- Input borders: #D1D5DB
- Sidebar divider: #F0F0F0

Difficulty badge colors:

- Easy: green background #DCFCE7, text #15803D
- Moderate: orange background #FFF7ED, text #C2410C
- Challenging: red background #FEF2F2, text #B91C1C

Typography

Font family: Inter — this is unmistakably Inter. Import from Google Fonts:

<https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap>

Then in Tailwind config: `fontFamily: { sans: ['Inter', 'sans-serif'] }`

Font sizes and weights across the UI:

Element	Size	Weight	Color
VedaAI logo text	18px	700	#111111
Sidebar nav items	14px	400	#6B6B6B
Sidebar active nav item	14px	500	#111111
"Create Assignment" button	14px	500	#FFFFFF
Page title ("Assignments")	20px	600	#111111
Page subtitle	13px	400	#6B6B6B
Card title	16px	600	#111111
Card meta text ("Assigned on")	13px	400	#6B6B6B
Card meta value (date)	13px	500	#111111
Form section title ("Assignment Details")	16px	600	#111111
Form section subtitle	13px	400	#6B6B6B
Input placeholder	14px	400	#9CA3AF
Input value	14px	400	#111111
Question type dropdown	14px	400	#111111
Total counts text	13px	400/500	#111111
Output: school name	22px	700	#111111
Output: subject/class	15px	400	#111111
Output: section header	18px	600	#111111
Output: question text	14px	400	#111111
Difficulty tag text	11px	500	(per color)
Dark banner text	14px	400	#FFFFFF
"Download as PDF" button text	13px	500	#FFFFFF

Empty state heading	18px	600	#111111
Empty state subtext	14px	400	#6B6B6B

Sidebar

Desktop sidebar dimensions: 200px wide, full height, white background, right border 1px solid #F0F0F0.

Logo area (top):

- Height: ~64px, padding: 16px 20px
- Logo mark: the orange square with rounded corners (~32px) containing a white stylized "V" shape — this is a custom SVG, not a library icon. Build it as an inline SVG with `fill="#E8470A"` background rect with `rx="6"` and a white path inside.
- "VedaAI" text: 18px, weight 700, color #111111, margin-left 8px

"+ Create Assignment" button:

- Full width minus 20px padding each side
- Height: 40px
- Background: #111111 (dark, near black — NOT the orange)
- Border radius: 8px
- Text: "◆ Create Assignment" — the ◆ sparkle icon before the text, white, 14px, weight 500
- The sparkle (◆) is a unicode character ◆ (U+2726) or use a small SVG star

Nav items:

- Padding: 10px 16px
- Border radius: 8px
- Icon + label, gap 10px
- Icons: 18px, from Lucide React (Home, Users, FileText, Wand2, BookOpen)
- Active state: background #F5F5F5, text color #111111, weight 500
- Inactive state: no background, text color #6B6B6B
- "Assignments" nav item shows an orange badge #E8470A with white number text, 11px, positioned as a pill on the right side of the row

Bottom section (separated by generous margin-top: auto):

- Settings row: gear icon + "Settings" text, 14px, #6B6B6B
- Teacher profile card:

- White background, border 1px solid #E5E7EB, border-radius 10px
 - Padding: 12px
 - Avatar: 36px circle, contains an illustrated avatar (cartoonish teacher illustration — these are custom SVG avatars, not initials. Use DiceBear Avatars library or similar:
<https://api.dicebear.com/7.x/avataaars/svg?seed=DelhiPublicSchool>)
 - School name: 13px, weight 600, #111111
 - City: 12px, weight 400, #6B6B6B
-

Top Navigation Bar

Height: 56px

Background: #FFFFFF

Border bottom: 1px solid #E5E7EB

Padding: 0 24px

Layout: flex, space-between

Left side:

- Back arrow icon (Lucide [ChevronLeft](#) or [ArrowLeft](#)), 18px, #6B6B6B
- Breadcrumb icon (grid/layout icon, Lucide [LayoutGrid](#)), 14px, #6B6B6B
- Breadcrumb text: "Assignment", 14px, #6B6B6B

Right side:

- Bell icon (Lucide [Bell](#)), 20px, #6B6B6B, with a small red dot notification badge (8px circle, #E8470A, positioned absolute top-right of the bell)
 - Avatar circle: 32px, circular image of teacher
 - Teacher name: "John Doe", 14px, weight 500, #111111
 - Chevron down: 14px, #6B6B6B
-

Screen 1 — Zero State (Assignments Page)

Empty state container: centered vertically and horizontally in the content area.

Illustration: The magnifying glass with red X and document — this is NOT from a library. It's a custom SVG illustration. Build it yourself or use an open-source illustration library like [unDraw](#) (search "empty" or "no data") or [Storyset](#). The color scheme of the illustration uses soft

purple/lavender for the document and glass, red for the X. Alternatively use the Lucide [SearchX](#) icon scaled up to ~120px with a light gray background circle.

"No assignments yet": 18px, weight 600, [#111111](#), margin-top 24px

Subtext: 14px, weight 400, [#6B6B6B](#), max-width 300px, text-align center, line-height 1.6

"+ Create Your First Assignment" button:

- Background: [#111111](#)
 - Text: white, 14px, weight 500
 - Padding: 12px 24px
 - Border radius: 8px
 - "+" prefix character
-

Screen 2 — Filled State (Assignments Grid)

Page header:

- Green dot indicator (8px circle, [#22C55E](#)) before "Assignments" title — indicates active/live status
- Title: "Assignments", 20px, weight 600
- Subtitle: "Manage and create assignments for your classes.", 13px, [#6B6B6B](#)

Filter bar:

- Full width, flex row, space-between
- Left: "Filter By" with filter icon (Lucide [SlidersHorizontal](#)), 14px, [#6B6B6B](#), no background, just text+icon
- Right: Search input — height 36px, border [1px solid #E5E7EB](#), border-radius 8px, padding 0 12px, placeholder "Search Assignment", Lucide [Search](#) icon inside on left, 14px

Assignment cards:

- Grid: 2 columns on desktop ([grid-template-columns: 1fr 1fr](#)), gap 12px
- Card: white background, border [1px solid #E5E7EB](#), border-radius 12px, padding 16px 20px
- Card height: approximately 100px
- Title: 16px, weight 600, [#111111](#), margin-bottom auto (pushed to top)
- Three-dot menu: Lucide [MoreVertical](#), 16px, [#9CA3AF](#), positioned top-right of card
- Bottom row: flex, space-between

- "Assigned on : " label 13px weight 400 #6B6B6B + date value 13px weight 500 #111111
- "Due : " label 13px weight 400 #6B6B6B + date value 13px weight 500 #111111

Three-dot dropdown menu:

- White background, border 1px solid #E5E7EB, border-radius 8px
- Shadow: box-shadow: 0 4px 12px rgba(0,0,0,0.08)
- Two items: "View Assignment" (14px, #111111) and "Delete" (14px, #EF4444 red)
- Padding: 8px 0, each item 8px 16px

Floating "+ Create Assignment" button:

- Positioned: bottom center, position: sticky at bottom of scroll area
- Same dark style as sidebar CTA but with + prefix
- Shadow: box-shadow: 0 4px 16px rgba(0,0,0,0.15)

Screen 3 — Create Assignment Form

Step indicator:

- Full width bar at top of content card
- Two steps indicated by a horizontal progress line
- Active step: thick dark line #111111, height 3px, border-radius 2px
- Inactive step: light line #E5E7EB, same dimensions
- The bar appears to be split into two equal halves with a gap between them

Content card:

- White background
- Border: 1px solid #E5E7EB
- Border radius: 16px
- Padding: 32px
- Max-width approximately 560px, centered

"Assignment Details" section:

- Title: 16px, weight 600, #111111
- Subtitle: 13px, #6B6B6B, margin-bottom 16px

File upload zone:

- Border: 2px dashed #D1D5DB
- Border radius: 12px
- Background: #FAFAFA
- Padding: 32px
- Center-aligned: upload cloud icon (Lucide UploadCloud), 28px, #9CA3AF
- "Choose a file or drag & drop it here" — 14px, weight 500, #111111
- "JPEG, PNG, upto 10MB" — 12px, #9CA3AF
- "Browse Files" button: border 1px solid #D1D5DB, background white, border-radius 6px, padding 8px 16px, 13px, weight 500, #111111
- Below the zone: helper text "Upload images of your preferred document/image", 12px, #9CA3AF, centered

Due Date input:

- Label: "Due Date", 14px, weight 500, #111111, margin-bottom 6px
- Input: height 40px, border 1px solid #D1D5DB, border-radius 8px, padding 0 12px, full width
- Placeholder "DD-MM-YYYY", 14px, #9CA3AF
- Right icon: calendar icon (Lucide CalendarPlus), 16px, #9CA3AF

Question Type section:

- Label row: "Question Type" | "No. of Questions" | "Marks" — 13px, weight 500, #6B6B6B, flex row with space-between, margin-bottom 8px

Each question type row:

- Height: 44px
- Background: white, border 1px solid #E5E7EB, border-radius 8px
- Layout: flex row, aligned center
- Left: dropdown select (takes ~50% width), Chevron down icon, 14px #111111
- Middle: × delete button — Lucide X, 14px, #9CA3AF, clickable
- Right two sections: stepper for count + stepper for marks
 - Each stepper: − button | number | + button
 - Stepper buttons: 28px circle or square, border 1px solid #E5E7EB, font 16px, color #111111
 - Number display: 14px, weight 500, #111111, min-width 24px, text-align center

" + Add Question Type" row:

- + icon in a circle (28px, border 1px solid #111111, color #111111)
- "Add Question Type" text, 14px, weight 500, #111111

- No background, just the icon and text

Totals row:

- Aligned right
- "Total Questions : 25" — 13px, weight 400, #111111 (value weight 600)
- "Total Marks : 60" — same style
- Line-height tight, two separate lines

Additional Information textarea:

- Label: "Additional Information (For better output)", 14px, weight 500, #111111
- Textarea: min-height 80px, border 1px solid #E5E7EB, border-radius 8px, padding 12px
- Placeholder: 13px, #9CA3AF, italic
- Microphone icon bottom-right corner of textarea: Lucide Mic, 16px, #9CA3AF

Navigation buttons (bottom):

- Left: "← Previous" — outlined button, border 1px solid #D1D5DB, background white, border-radius 8px, padding 10px 24px, 14px, weight 500, #111111, left arrow
- Right: "Next →" — filled button, background #111111, text white, same sizing, right arrow

Screen 4 — Assignment Output

Dark banner:

- Background: #1A1A1A
- Border radius: 12px (top of the paper section)
- Padding: 20px 24px
- Text: 14px, weight 400, white, line-height 1.5, max-width ~600px
- "Download as PDF" button:
 - Border: 1px solid rgba(255,255,255,0.3)
 - Background: transparent (outlined style)
 - Text: white, 13px, weight 500
 - Icon: Lucide Download, 14px, white
 - Border radius: 8px
 - Padding: 8px 16px

Exam paper (white card):

- Background: #FFFFFF
- Border: 1px solid #E5E7EB
- Border radius: 0 0 12px 12px (bottom radius only, connects to dark banner above)
- Padding: 40px 48px (generous, like a real printed paper)
- Max-width: ~700px

Paper content typography:

- School name: 20px, weight 700, #111111, text-align center
- Subject + Class: 15px, weight 400, #111111, text-align center
- Time Allowed + Max Marks: 13px, weight 400, #111111, flex row space-between
- Horizontal rule: border-top: 1px solid #E5E7EB
- General instruction: 13px, weight 500, #111111, italic
- Name/Roll/Section lines: 13px, weight 400, #111111, with underscores rendered as border-bottom: 1px solid #111111 on a fixed-width span
- Section header ("Section A"): 16px, weight 700, #111111, text-align center
- Section type name ("Short Answer Questions"): 14px, weight 600, #111111
- Section instruction ("Attempt all questions..."): 13px, italic, #6B6B6B
- Question text: 14px, weight 400, #111111, line-height 1.6
- Difficulty tag: inline, [Easy] / [Moderate] / [Challenging] — in the Figma these appear as plain text brackets, NOT as colored badges. But the assignment asks for visual highlighting, so add small colored pills
- Marks: [2 Marks] — 13px, #6B6B6B, at end of question line

"End of Question Paper": 14px, weight 600, #111111, text-align left, margin-top 24px

Answer Key: 14px, weight 600, #111111, section below a divider line. Answers as numbered list, 13px, #333333

Mobile (Dashboard column in screenshots)

Top bar:

- VedaAI logo left
- Bell icon + avatar + hamburger menu icon right
- Height: 52px

Bottom tab bar:

- Background: #111111 (dark)
- 4 tabs: Home, Assignments, Library, AI Toolkit

- Active tab: white icon + white text, small white indicator dot above
- Inactive tab: grey icon + grey text
- Height: 64px

Cards on mobile: Full width, single column, same styling as desktop cards but width 100%

Form on mobile: Question type rows stack — the No. of Questions and Marks steppers drop below the dropdown selector

UI Library Recommendations

Based on what I see, use these exact libraries:

Component library: [shadcn/ui](#) — it matches the exact aesthetic (clean, minimal, Inter font, consistent border radius, no heavy opinionated styling). Install individual components as needed.

Icons: [lucide-react](#) — every icon in this UI is from Lucide. 100% match.

Avatars: [DiceBear](#) ([@dicebear/core](#) + [@dicebear/collection](#)) — for the cartoonish teacher avatar in the sidebar profile card.

Date picker: [react-day-picker](#) (used by shadcn's Calendar component) — matches the DD-MM-YYYY input style.

Drag and drop file upload: [react-dropzone](#) — for the dashed upload zone.

Tailwind config additions needed:

```
js
theme: {
  extend: {
    fontFamily: { sans: ['Inter', 'sans-serif'] },
    colors: {
      brand: '#E8470A',
      'brand-dark': '#C73D08',
    }
  }
}
```

VedaAI — Mobile Responsive Implementation Guide

CORE PRINCIPLE

This is ONE Next.js codebase. No separate mobile app. The same components render differently at different screen widths using Tailwind breakpoints. The Figma mobile screenshots show exactly what each screen should look like below 1024px.

Breakpoint reference:

< 768px → mobile phones

768–1023px → tablets / small laptops

1024px+ → desktop / large laptops ← your primary target

LAYOUT CHANGES AT EACH BREAKPOINT

THE BIGGEST CHANGE — Sidebar vs Bottom Tab Bar

Desktop (lg+):

- Left sidebar is visible: w-[200px], fixed, full height
- Main content has: ml-[200px]
- Bottom tab bar: HIDDEN

Mobile (< lg):

- Sidebar is HIDDEN completely
- Main content has: ml-0, pb-16 (padding bottom so content doesn't hide behind tab bar)
- Bottom tab bar: FIXED at bottom, full width, height 64px, dark background #111111

// DashboardLayout.tsx structure

```
<div className="min-h-screen bg-gray-50">
```

```
  {/* Sidebar — desktop only */}
```

```
  <aside className="hidden lg:flex flex-col fixed left-0 top-0 h-full w-[200px]
    bg-white border-r border-gray-100 z-40">
```

```
    <Sidebar />
```

```

</aside>

{/* Top bar — always visible */}
<header className="fixed top-0 right-0 left-0 lg:left-[200px] h-14
    bg-white border-b border-gray-200 z-30">
  <TopBar />
</header>

{/* Main content */}
<main className="pt-14 lg:ml-[200px] pb-16 lg:pb-0 min-h-screen">
  <div className="max-w-[1200px] mx-auto px-4 lg:px-6 py-6">
    {children}
  </div>
</main>

{/* Bottom tab bar — mobile only */}
<nav className="lg:hidden fixed bottom-0 left-0 right-0 h-16
    bg-[#111111] z-40 flex items-center justify-around px-4">
  <BottomTabBar />
</nav>

</div>

```

COMPONENT-BY-COMPONENT MOBILE SPEC

1. TopBar

Desktop:

- Left: back arrow + breadcrumb icon + "Assignment" text
- Right: bell icon + avatar + name + chevron

Mobile:

- Left: VedaAI logo (icon + text)
- Right: bell icon + avatar + hamburger menu (lucide Menu icon)
- No breadcrumb text on mobile
- Teacher name hidden on mobile

```

<header>
  <div className="flex items-center justify-between h-14 px-4">

```

```

    { /* Left — logo on mobile, breadcrumb on desktop */}
    <div className="flex items-center gap-2">
      <span className="lg:hidden"><VedaLogo /></span>
      <span className="hidden lg:flex items-center gap-2 text-gray-500 text-sm">
        <ChevronLeft size={16} />
        <LayoutGrid size={14} />
        Assignment
      </span>
    </div>

    { /* Right */}
    <div className="flex items-center gap-3">
      <button className="relative">
        <Bell size={20} className="text-gray-500" />
        <span className="absolute top-0 right-0 w-2 h-2 bg-brand-orange rounded-full" />
      </button>
      <img className="w-8 h-8 rounded-full" src={avatarUrl} />
      { /* Name — desktop only */}
      <span className="hidden lg:block text-sm font-medium text-gray-900">John Doe</span>
      <ChevronDown size={14} className="hidden lg:block text-gray-500" />
      { /* Hamburger — mobile only */}
      <button className="lg:hidden">
        <Menu size={20} className="text-gray-500" />
      </button>
    </div>

  </div>
</header>

```

2. Bottom Tab Bar (mobile only)

4 tabs: Home, Assignments, Library, AI Toolkit
 Active tab: white icon + white text + small white dot above
 Inactive tab: gray-400 icon + gray-400 text

```

// BottomTabBar.tsx
const tabs = [
  { label: 'Home', icon: Home, href: '/dashboard' },
  { label: 'Assignments', icon: FileText, href: '/assignments' },
  { label: 'Library', icon: BookOpen, href: '/library' },
  { label: 'AI Toolkit', icon: Wand2, href: '/toolkit' },
]

```



```

export function BottomTabBar() {
  const pathname = usePathname()

  return (
    <>
      {tabs.map(tab => {
        const active = pathname.startsWith(tab.href)
        return (
          <Link key={tab.href} href={tab.href}
            className="flex flex-col items-center gap-1 flex-1">
              {/* Active indicator dot */}
              <span className={`w-1 h-1 rounded-full mb-0.5
                ${active ? 'bg-white' : 'bg-transparent'}}` />
              <tab.icon size={20}
                className={active ? 'text-white' : 'text-gray-400'} />
              <span className={`text-[10px]
                ${active ? 'text-white' : 'text-gray-400'}}`>
                {tab.label}
              </span>
            </Link>
          )
        })}
    </>
  )
}

```

3. Assignments List Page

Desktop: 2-column card grid Mobile: 1-column card grid, filter bar stacks vertically

```

{/* Page header */}
<div className="mb-6">
  <div className="flex items-center gap-2 mb-1">
    <span className="w-2 h-2 rounded-full bg-green-500" />
    <h1 className="text-xl font-semibold text-gray-900">Assignments</h1>
  </div>
  <p className="text-sm text-gray-500">Manage and create assignments for your classes.</p>
</div>

{/* Filter bar — stacks on mobile */}
<div className="flex flex-col sm:flex-row gap-3 mb-6">
  <button className="flex items-center gap-2 text-sm text-gray-500">

```

```

    <SlidersHorizontal size={14} />
    Filter By
  </button>
  <div className="relative flex-1">
    <Search size={14} className="absolute left-3 top-1/2 -translate-y-1/2 text-gray-400" />
    <input className="w-full h-9 pl-8 pr-4 border border-gray-200 rounded-lg text-sm"
      placeholder="Search Assignment" />
  </div>
</div>

{/* Grid — 2 col desktop, 1 col mobile */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-3">
  {assignments.map(a => <AssignmentCard key={a._id} assignment={a} />)}
</div>

{/* Floating create button */}
<div className="fixed bottom-20 lg:bottom-6 left-1/2 -translate-x-1/2 z-20">
  <button className="flex items-center gap-2 bg-[#111111] text-white
    text-sm font-medium px-5 py-2.5 rounded-full shadow-xl">
    <Plus size={16} />
    Create Assignment
  </button>
</div>

```

Note: **bottom-20** on mobile (above the tab bar), **bottom-6** on desktop.

4. Create Assignment Form

Desktop: form card centered with max-w-[600px] Mobile: form fills full width, question type rows stack

```

{/* Form wrapper */}
<div className="max-w-[600px] mx-auto w-full">

  {/* Step progress bar */}
  <div className="flex gap-2 mb-8">
    <div className={`h-[3px] flex-1 rounded-full
      ${currentStep >= 1 ? 'bg-[#111111]' : 'bg-gray-200'} ` />
    <div className={`h-[3px] flex-1 rounded-full
      ${currentStep >= 2 ? 'bg-[#111111]' : 'bg-gray-200'} ` />
  </div>

```

```

    { /* Form card */}
    <div className="bg-white border border-gray-200 rounded-2xl p-5 lg:p-8">
      ...
    </div>

  </div>

```

Question type row — stacks on mobile:

```

{ /* Desktop: all inline. Mobile: dropdown full width, steppers below */}
<div className="border border-gray-200 rounded-lg p-3">

  { /* Row 1: dropdown + delete button */}
  <div className="flex items-center gap-2 mb-3 lg:mb-0">
    <select className="flex-1 lg:w-[45%] h-9 border border-gray-200
      rounded-lg px-3 text-sm">
      {questionTypeOptions.map(opt => (
        <option key={opt}>{opt}</option>
      ))}
    </select>
    <button onClick={() => removeRow(row.id)}>
      <X size={14} className="text-gray-400" />
    </button>
  </div>

  { /* Row 2 on mobile, inline on desktop */}
  <div className="flex items-center gap-4 lg:ml-auto">
    <div className="flex-1 lg:flex-none">
      <p className="text-xs text-gray-500 mb-1 lg:hidden">No. of Questions</p>
      <Stepper value={row.count} onChange={v => updateRow(row.id, 'count', v)} />
    </div>
    <div className="flex-1 lg:flex-none">
      <p className="text-xs text-gray-500 mb-1 lg:hidden">Marks</p>
      <Stepper value={row.marksEach} onChange={v => updateRow(row.id, 'marksEach', v)} />
    </div>
  </div>

</div>

```

Navigation buttons — full width on mobile:

```

<div className="flex gap-3 mt-6">
  <button className="flex-1 lg:flex-none lg:w-auto h-10 border border-gray-300

```

```

        rounded-lg text-sm font-medium text-gray-900 px-6">
    <← Previous
</button>
<button className="flex-1 lg:flex-none lg:w-auto h-10 bg-[#111111]
        rounded-lg text-sm font-medium text-white px-6 ml-auto">
    Next →
</button>
</div>

```

5. Output Page

Desktop: dark banner + white paper card, max-w-[720px], generous padding
 Mobile: same structure, reduced padding, font sizes scale down

```

<div className="max-w-[720px] mx-auto w-full">

  {/* Dark banner */}
  <div className="bg-[#1A1A1A] rounded-t-xl p-4 lg:p-6">
    <p className="text-white text-sm leading-relaxed mb-3">
      Certainly, Lakshya! Here are customized Question Paper...
    </p>
    <button className="flex items-center gap-2 border border-white/30
        text-white text-xs font-medium px-3 py-2 rounded-lg">
      <Download size={13} />
      Download as PDF
    </button>
  </div>

  {/* Exam paper */}
  <div className="bg-white border border-gray-200 rounded-b-xl
        px-4 py-6 lg:px-12 lg:py-10">

    {/* School header */}
    <h1 className="text-lg lg:text-2xl font-bold text-center text-gray-900">
      Delhi Public School, Bokaro
    </h1>
    <p className="text-sm lg:text-base text-center text-gray-700 mt-1">
      Subject: English
    </p>
    <p className="text-sm lg:text-base text-center text-gray-700">
      Class: 5th
    </p>
  </div>

```

```
<hr className="border-gray-200 my-4" />
```

```
{/* Time + Marks row */}
```

```
<div className="flex justify-between text-xs lg:text-sm text-gray-900">
```

```
  <span>Time Allowed: 45 minutes</span>
```

```
  <span>Maximum Marks: 20</span>
```

```
</div>
```

```
<hr className="border-gray-200 my-4" />
```

```
{/* Instructions */}
```

```
<p className="text-xs lg:text-sm font-medium text-gray-900 mb-4">
```

```
  All questions are compulsory unless stated otherwise.
```

```
</p>
```

```
{/* Student info */}
```

```
<div className="space-y-2 mb-6">
```

```
  <p className="text-xs lg:text-sm text-gray-900">
```

```
    Name: <span className="inline-block w-32 lg:w-48 border-b border-gray-900 ml-1" />
```

```
  </p>
```

```
  <p className="text-xs lg:text-sm text-gray-900">
```

```
    Roll Number: <span className="inline-block w-24 lg:w-36 border-b border-gray-900 ml-1"
```

```
/>
```

```
  </p>
```

```
  <p className="text-xs lg:text-sm text-gray-900">
```

```
    Class/Section: <span className="inline-block w-20 lg:w-28 border-b border-gray-900
```

```
ml-1" />
```

```
  </p>
```

```
</div>
```

```
<hr className="border-gray-200 mb-6" />
```

```
{/* Sections */}
```

```
{paper.sections.map(section => (
```

```
  <SectionBlock key={section.id} section={section} />
```

```
)))
```

```
<hr className="border-gray-200 my-6" />
```

```
<p className="text-sm font-semibold text-gray-900">End of Question Paper</p>
```

```
</div>
```

```
</div>
```

Question item — difficulty badge and marks wrap on mobile:

```
<div className="flex items-start gap-2 py-2">
  <span className="text-sm text-gray-900 min-w-[1.5rem]">{question.id}</span>
  <div className="flex-1">
    <div className="flex flex-wrap items-start gap-2">
      <DifficultyBadge difficulty={question.difficulty} />
      <p className="text-sm text-gray-900 flex-1 min-w-[200px]">
        {question.text}
      </p>
      <span className="text-xs text-gray-500 whitespace-nowrap ml-auto">
        [{question.marks} Marks]
      </span>
    </div>
    {question.options && (
      <div className="grid grid-cols-2 gap-1 mt-2 ml-0">
        {question.options.map((opt, i) => (
          <span key={i} className="text-sm text-gray-900">
            ({String.fromCharCode(97 + i))} {opt}
          </span>
        ))}
      </div>
    )}
  </div>
</div>
```

6. Generating / Loading Page

Same on all screen sizes — just a centered loading state:

```
<div className="min-h-screen flex items-center justify-center px-4">
  <div className="text-center max-w-sm w-full">
    <div className="w-16 h-16 border-4 border-gray-200 border-t-brand-orange
      rounded-full animate-spin mx-auto mb-6" />
    <h2 className="text-lg font-semibold text-gray-900 mb-2">
      Generating your assessment
    </h2>
    <p className="text-sm text-gray-500">{statusMessage}</p>
    <div className="mt-6 w-full bg-gray-200 rounded-full h-1.5">
      <div className="bg-brand-orange h-1.5 rounded-full transition-all duration-500"
        style={{ width: `${progress}%` }} />
    </div>
  </div>
</div>
```

</div>

7. Empty State (Zero state)

Same layout on all screens, just padding changes:

```
<div className="flex flex-col items-center justify-center
  min-h-[60vh] text-center px-4">

  {/* Illustration — use Lucide SearchX in a circle */}
  <div className="w-24 h-24 lg:w-32 lg:h-32 rounded-full bg-gray-100
    flex items-center justify-center mb-6">
    <SearchX size={40} className="text-gray-400" />
  </div>

  <h2 className="text-lg font-semibold text-gray-900 mb-2">
    No assignments yet
  </h2>
  <p className="text-sm text-gray-500 max-w-[280px] leading-relaxed mb-6">
    Create your first assignment to start collecting and grading student submissions.
  </p>
  <button className="flex items-center gap-2 bg-[#111111] text-white
    text-sm font-medium px-5 py-2.5 rounded-lg">
    <Plus size={16} />
    Create Your First Assignment
  </button>

</div>
```

WHAT TO ADD TO EVERY STEP PROMPT IN CLAUDE CODE

At the end of every step prompt you give Claude Code, add this line:

Also apply mobile responsive classes throughout:

- Sidebar hidden on mobile (lg:flex hidden), bottom tab bar visible only on mobile (lg:hidden)
- Main content ml-[200px] on lg+, ml-0 on mobile with pb-16 for tab bar
- Cards single column on mobile (grid-cols-1 lg:grid-cols-2)
- Form question type rows stack on mobile (flex-col lg:flex-row)

- Floating button bottom-20 on mobile (above tab bar), bottom-6 on desktop
 - Reduced padding on mobile (px-4 lg:px-12) for paper and cards
- Reference the mobile column in docs/screenshots/ for exact visual target.

That one paragraph added to each step prompt is all you need.

QUICK REFERENCE — MOST USED RESPONSIVE PATTERNS

Sidebar visible:	hidden lg:flex
Bottom tab bar:	lg:hidden fixed bottom-0
Main content offset:	lg:ml-[200px]
Main content padding:	pb-16 lg:pb-0
Grid columns:	grid-cols-1 lg:grid-cols-2
Floating btn position:	bottom-20 lg:bottom-6
Paper padding:	px-4 py-6 lg:px-12 lg:py-10
Form card padding:	p-5 lg:p-8
Question row layout:	flex-col lg:flex-row
Font size scale:	text-lg lg:text-2xl (school name) text-xs lg:text-sm (meta text)
Nav name visible:	hidden lg:block
Hamburger visible:	lg:hidden